

Zephyr RTOS 入門-簡介及代碼下載、編譯、運行

2024-01-08748 閱讀 23 分鐘

說到 RTOS，第一印象就是單片機程式、ARM M 核心、微控制器用於處理低頻工作的程式，甚至不能被視為真正的操作系統。但隨著時代的發展，這些單片機程式在有限的硬體環境下，已經能夠媲美 Linux 這樣的大型操作系統了。它們具有線程、中斷處理、記憶體管理、IPC 通訊、驅動程式、網路協定堆疊、命令列介面，還提供 POSIX 接口等功能。可以說，它們已經是“縮小版的操作系統”了。甚至，它們也能在 A 核心上運行，只要稍作修改，加上一些用戶空間相關的功能即可。

在嵌入式領域中，如果需要的是經過定制化的軟硬件，且不需要任何界面展示，那麼就不需要像 ARM A 核這樣的高級 CPU。M 核是一個更好的選擇：它價格低廉、體積小、耗電量少、可靠性高，且更適合用於專用控制系統。不過，M 核的運行頻率較低，功能也有所限制。在 M 核上運行的 RTOS 也具有相應的特點：

1. 地址空間只有一個，就像 Linux 的內核一樣，共享地址空間。
2. 沒有高級的硬件相關 MMU 功能，無虛擬地址
3. 沒有檔案系統、UI 介面等複雜服務
4. 各種服務，例如進程調度等，都只是簡單的版本而已。

之前介紹過一個典型的 RTOS，那就是 freeRTOS。其實，RTOS 的定義就是實時操作系統。例如，只要對 Linux 進行一些修改，使其具備實時特性，那它也可以被稱為 RTOS。本文所介紹的 zephyr RTOS，就是一個類似 freeRTOS 的操作系統，可以作為 freeRTOS 的替代品。下面將介紹其特點，先直接上代碼：[github.com/zephyrproje...](https://github.com/zephyrproject-rtos)

1. Zephyr 簡介

1.1 Zephyr 簡介

Zephyr 最初是由 Wind River 公司开发的一个微内核。2016 年，它成为了由 Linux 基金会负责维护的项目。经过不断发展，它现已成为一款功能完备的嵌入式操作系统。该平台目前支持 ARM、RISC-V、X86、Xtensa 等各種處理器平台。它还拥有原生的 BLE 協議堆疊以及完整的網路協議堆疊，包括 TCP/IP 和各種應用層協議，从而為嵌入式應用的開發提供了有力的支持。

Zephyr 不僅維護了一個 RTOS 核心，還維護了各種編譯工具、libc、IDE 插件、HEL 層驅動等。幾乎每個模組都有相關的文檔。正因為有詳細的文檔，Zephyr 才能持續吸引人們進行嘗試與開發，從而讓系統不斷完善，進入開源項目的良性循環之中。

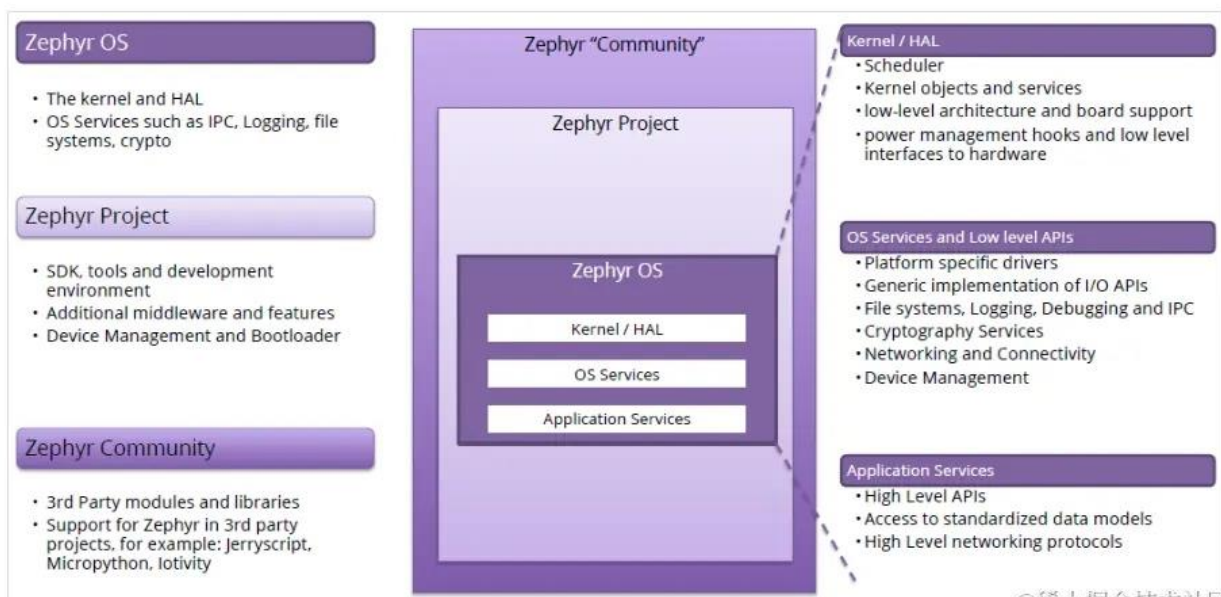
由 Linux 基金會維護

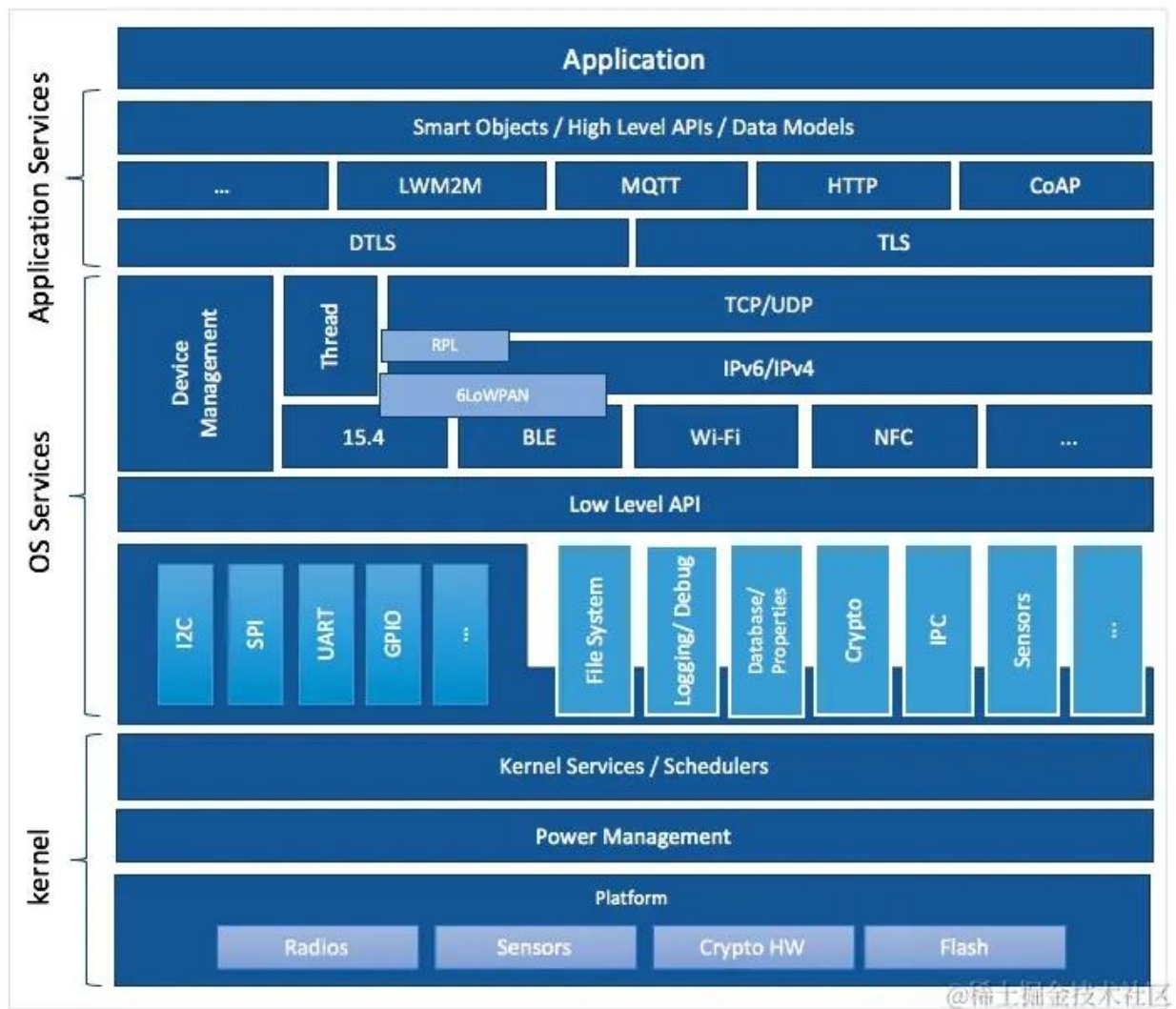
所以，Zephyr 給人的感覺就是“Linux 中的 Linux”：它支持 make menuconfig、DTS 等功能。可以說，Zephyr 將 Linux 的一些優秀特性都吸收進去了，許多代碼也與 Linux 保持同步。簡直就是 Linux 的“小兄弟”啊。好處是，當我們需要複製或移植代碼時，Zephyr 是一個非常好的研究對象——畢竟它完全是開源的啊！

Wind River 公司

舉世聞名的 VxWorks 即時作業系統是由 Wind River 公司所開發的。在即時作業系統領域，VxWorks 早在早期就處於領先地位。尤其在軍事領域，憑藉其出色的可靠性和即時性，它被廣泛應用於通訊、軍事、航空、航天等需要高精尖技術的領域，以及那些對即時性要求極高的場景，例如衛星通訊、軍事演習、彈道導引、飛機導航等。在美國的 F-16、FA-18 戰鬥機、B-2 隱形轟炸機以及“愛國者”導彈上，都使用了 VxWorks。此外，1997 年 4 月登陸火星表面的火星探測器、2008 年 5 月登陸的“鳳凰號”探測器，以及 2012 年 8 月登陸的“好奇號”探測器，也均採用了 VxWorks 作業系統。VxWorks 並非開源軟體，而 Zephyr 則可說是出身於同一個優秀的“家族”。Zephyr 的即時性特徵實在值得我們學習借鑑。

1.2 特性介紹





Zephyr OS 以佔用資源極低的核心為基礎，專為資源有限的嵌入式系統而設計：從簡單的嵌入式環境感測器、LED 可穿戴裝置，到複雜的嵌入式控制器、智慧手錶以及物聯網無線應用等。

其中，Zephyr OS 是 Zephyr 所使用的核心代碼，其包含以下內容：

- 核心/HAL
 - 內核調度
 - 低級的體系結構和板級支援
 - 各種 RTOS 核心對象
 - 電源管理框架
 - 低級的硬體介面抽象
- OS 服務及底層接口
 - 設備管理及底層驅動 API

- 檔案系統、記錄系統、除錯系統及 IPC
- 加密服務
- 與平台相關的特殊驅動程式
- 網路協議棧

- 應用服務

- 高層級的應用 API
- 標準化的數據模型
- 高層級網路協議接口

對於 Zephyr 應用開發者而言，需要熟悉應用服務和 OS 服務，了解底層接口以及 Kernel/HAL 的各項特性。

Zephyr 也提供了构建系统与开发环境。这些元素与 Zephyr OS 共同构成了 Zephyr Project。Zephyr Project 包含以下内容：

- Zephyr 的 SDK、建構工具、CI 及開發環境
- Bootloader：MCUboot
- 附加的中间件

除了核心代碼之外，Zephyr 還引用了其他優秀的開源項目作為其功能組成部分。同時，也有許多項目將 Zephyr 作為自己的組成部分。這些元素共同構成了完整的 Zephyr 生態系統。Zephyr 生態系統包含以下內容：

- 第三方模組或庫：例如 lvgl、fatfs、mcumgr、openthread、openamp、opencan 等等。
- 支持 Zephyr 的第三方項目：例如 Jerryscript、Micropython、Iotivity、SwiftIO 等。
- SOC 供應商 Hal 和 Zephyr 將不同芯片的 Hal 作為外部模組引入到 Zephyr 中。透過 Zephyr 所提供的驅動程式，各種應用程式都能透過統一的接口來控制不同芯片上的各種設備。例如：nxp、ST、TI、Nordic、Silabs 等等。

Kernel 包含以下內容：

- 提供核心服務，例如線程、同步、數據傳遞、中斷管理、時間管理、內存管理等。
- 進行任務調度
- 電源管理
- 與平台相關的專用驅動程式，例如無線收發模組、感測器、加密硬體、Flash 等。

OS 服務涵蓋以下內容：

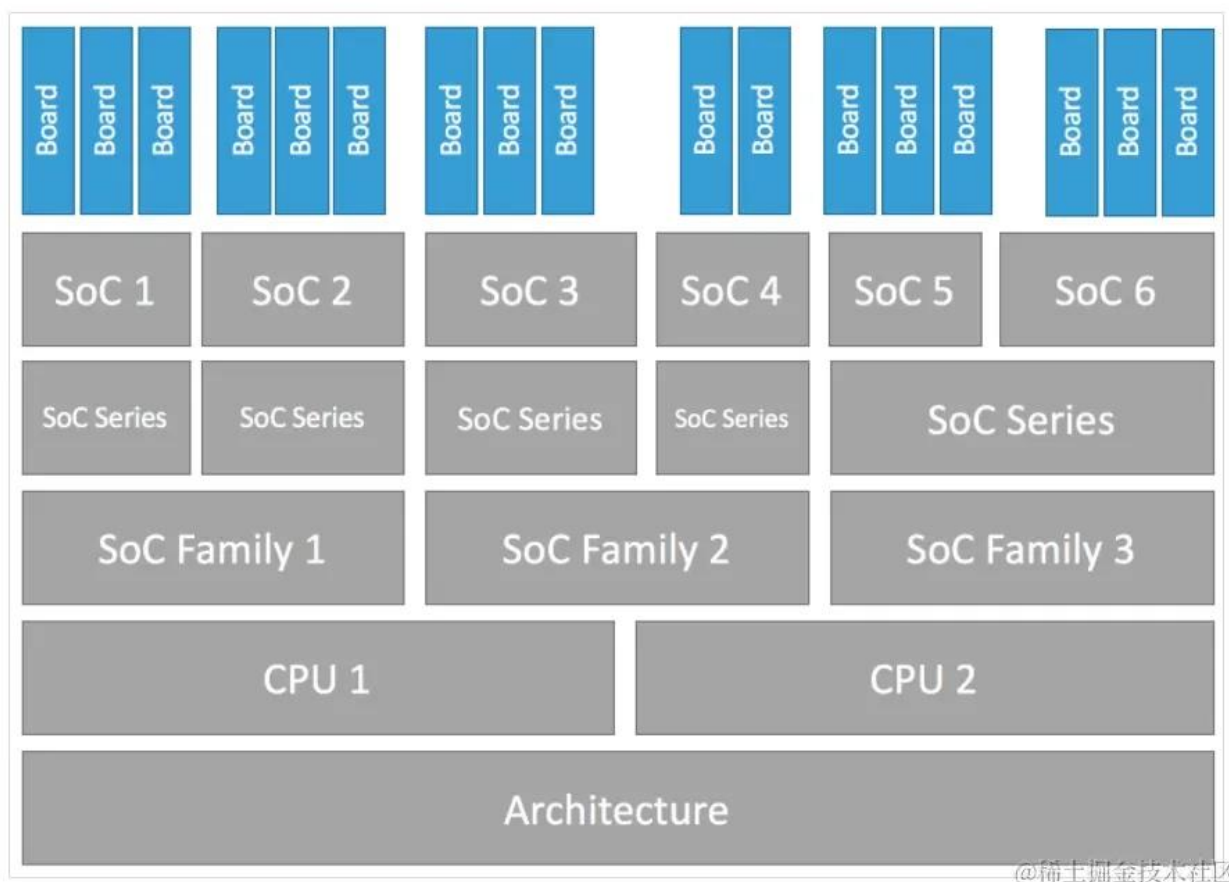
- 透過設備驅動程式實現，提供統一的底層驅動接口
- 設備管理

- 網路 L2 實現及介面抽象
- 網路層和傳輸層協議棧(TCP/IP)，socket 介面

應用服務涵蓋以下內容：

- 網路應用協議及高層級介面，例如 HTTP、CoAP、MQTT、TLS、DTLS 等
- 提供標準化的數據模型，目前尚未在 Zephyr 的目錄樹中找到該部分內容
- 提供智能物件相關的服務/功能，例如基於 lwm2m 的 IPSO。

Zephyr按照下面架构对不同硬件进行支持



Zephyr 的硬件層次結構被抽象出 6 層：

- 架構(Architecture)：指令架構體系，例如 ARM、RISC-V、x86 等
- CPU 核心：架构中特定的 CPU。例如，在 ARM 架构中，有 Cortex-M0、M3、M4、M7 等不同型号的 CPU 核心。
- 芯片族(Soc family)：具有相似特性的 SoC，例如 Cortex-M7 系列中就有 STMicro 的 STM32 和 NXP 的 i.MX 系列芯片。
- 芯片系列(SoC 系列)：由若干紧密相关的 SoC 组成的系列。例如，在 i.MX 系列中，就包括 i.MX RT 系列、i.MX 8 系列等。
- 芯片級(SoC)：電路板上的 SoC，例如 i.MX RT 系列中有 RT1050、RT1060 等芯片

- 板級：PCB 上，特定的 SoC 與各種外設相連後，便構成了具有特定功能的電路板。例如，Zephyr 所支援的 `mimxrt1050_evk`，以及 `mm_swiftio` 開發板，都使用了 `rt1052` 芯片。

Zephyr 核心支持多種架構，包括：

- ARCV2 (EM 和 HS) 以及 ARCV3 (HS6X)
- ARMv6-M、ARMv7-M 和 ARMv8-M (Cortex-M)
- ARMv7-A 和 ARMv8-A (Cortex-A 系列，32 位和 64 位)
- ARMv7-R、ARMv8-R (Cortex-R 系列，32 位和 64 位)
- Intel x86 (32 位和 64 位)
- MIPS (MIPS32 Release 1 規格)
- NIOS II 第 2 代
- RISC-V (32 位和 64 位)
- SPARC V8
- Tensilica Xtensa

Zephyr 擁有完善的核心服務套件，提供了許多大家熟悉的開發功能：

- ****這是一種用於協作的多線程服務，支持基於優先級的處理方式、非搶佔式及搶佔式線程。同時，還提供可選的循環時間切分功能。該服務兼容 POSIX pthreads API。****
- 中斷服務用於中斷處理程序的編譯時註冊。
- ****用於動態分配和釋放固定大小或可變大小內存塊的內存分配服務。**
- ****用於二進位信號量、計數信號量及互斥信號量的線程間同步服務。**
- ****用於基本消息隊列、增強消息隊列和字節流的線程間數據傳遞服務。**

Zephyr 提供了全方位的线程调度选项：

- 協作與搶佔式調度
- 最早截止日期優先 (EDF)
- 元 IRQ 調度實現“中斷下半部分”或“tasklet”行為
- 時間切片：在同等優先級的可搶佔線程之間啟用時間切片
- 多種排隊策略：
 - 簡單鏈表就緒隊列
 - 紅/黑樹就緒隊列
 - 傳統多隊列就緒隊列

高度可配置/模組化，以實現靈活性

- 允許應用程式僅根據需要合併其所需的機能，並指定其數量和大小。

跨架構

- 支援各種具有不同 CPU 架構及開發工具的主板。此外，也新增了對越來越多 SoC、平台及驅動程式的支援。

內存保護

- 在 x86、ARC 和 ARM 架构、用户空间和内存域上，可实现可配置的、特定于各架构的堆栈溢出保护机制；同时，还能对内核对象和设备驱动程序的权限进行跟踪，并实现具有线程级内存保护的线程隔离功能。

對於沒有 MMU/MPU 以及內存有限的設備，該技術支持將特定於應用程式的代碼與自訂核心相結合，從而創建出可在系統硬體上載入並執行的完整映像。應用程式代碼與核心代碼都在單一的共享地址空間中執行。

編譯時資源定義

- 允許在編譯時定義系統資源，從而減少代碼大小並提高資源有限的系統的性能。

優化的設備驅動模型

- 提供用於配置屬於平台/系統的驅動程式的統一設備模型，以及用於初始化配置到系統中的所有驅動程式的統一模型，並允許在具有公共設備/IP 位址的平台之間重用驅動程式。

設備樹支援

- 使用 devicetree 來描述硬件。來自 devicetree 的資訊用於創建應用程式映像。

支援多種協議的本機網路堆疊

- 該網路支援功能齊全且經過優化，包括對 LwM2M 和 BSD 套接字的兼容性支援。同時也提供 OpenThread 支援（適用於 Nordic 芯片組）——這是一種用於以安全可靠的方式，將家中的數百種設備連接起來的網狀網路技術。

低功耗藍牙 5.0 支持

- 符合藍牙 5.0 標準 (ESR10)，並支援藍牙低功耗控制器功能（LE 鏈路層）。包含藍牙網狀網路相關功能，以及符合藍牙相關規格的藍牙控制器。
 - 具有所有可能的 LE 角色的通用存取配置檔 (GAP)
 - 通用屬性配置檔案 (GATT)
 - 支持配對功能，包括藍牙 4.2 的安全連接功能
 - 乾淨的 HCI 驅動程式抽象層
 - 原始的 HCI 接口將 Zephyr 作為控制器來運行，而非完整的宿主系統堆疊。

- 經多個流行控制器驗證
- 高度可配置

網格支援：

- 中繼、朋友節點、低功耗節點 (LPN) 和 GATT 代理功能
- 支援兩種配置方式 (PB-ADV 和 PB-GATT)
- 高度可配置，適合至少擁有 16k RAM 的設備

本機支持 **Linux**、**macOS** 和 **Windows** 系統開發。

- 命令行 CMake 构建环境可在各种流行的开发者操作系统中运行。通过使用本机端口 (native_sim)，您可以在 Linux 上将 Zephyr 构建为本地应用程序并直接运行它，从而便于开发和测试工作。

支援 **ext2**、**FatFs** 和 **LittleFS** 的虛擬檔案系統介面

- 支援 ext2、LittleFS 和 FatFS；FCB（快閃記憶體循環緩衝區）適用於內存有限的應用程式。

強大的多後端日誌框架

- 支持日誌過濾、對象轉儲、恐慌模式、多個後端（內存、網路、檔案系統、控制台.....）以及與 shell 子系統的集成。

使用者友善且功能齊全的 **Shell** 介面

- 多實例 shell 子系統擁有使用者友善的各種功能，例如自動完成、通配符、顏色標記、特殊鍵功能（箭頭鍵、退格鍵、Ctrl+U 等）以及歷史記錄功能。同時也支援靜態命令和動態子命令。

非揮發性儲存上的設定

- 將子系統設定為模組，提供了一種用於儲存每個裝置的永久性配置及運行時狀態的方法。各項設定均以鍵值對的形式儲存。

非揮發性儲存 (**NVS**)

- NVS 允許儲存二進位 blob、字串、整數、長整型，以及這些類型的任意組合。

本機端口

- Native SIM 允許將 Zephyr 作為 Linux 應用程式來運行，並支持各種子系統和網絡。

2. 環境搭建與代碼下載編譯

參考：[docs.zephyrproject.org/latest/deve...](https://docs.zephyrproject.org/latest/devel) blog.csdn.net/shenhuxi_yu... 多種編譯工具的編譯參考：liao-xingyu.gitee.io/p/zephyr-%E...

請按照本指南進行操作：

- 在 Ubuntu、macOS 或 Windows 上設定命令列 Zephyr 開發環境（其他 Linux 發行版的說明在 安裝 Linux 主機依賴項中討論）
- 獲取源代碼
- 構建、刷新並運行示例應用程式

具體內容就不再詳述了。如果是在 QEMU 環境下運行，請參考以下說明：

[在 QEMU 中執行應用程式](#)

在 Linux 和 macOS 上，當面對 x86 或 ARM Cortex-M3 架構時，您可以使用 QEMU 在主机系統上透過仿真來運行 Zephyr 應用程式。（QEMU 已包含在 Zephyr SDK 的安裝包中。）

例如，您可以使用 x86 仿真板配置来构建并运行 Hello World `qemu_x86` 示例，其中：

```
# From the root of the zephyr repository
west build -b qemu_x86 samples/hello_world
west build -t run
```

若要退出 QEMU，請鍵入以下指令：`Ctrl-a`x`

用於 `qemu_cortex_m3` 中針對模擬環境的 Arm Cortex-M3 範例。

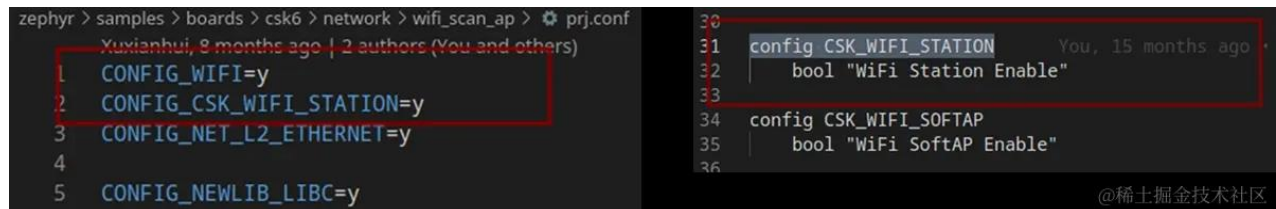
3. Zephyr 的內核特性

3.1 建構

ephyr 的构建系统主要由三个部分组成：cmake、kconfig、devicetree。

cmake：在初期阶段，需要利用 **kconfig** 和 **devicetree** 来生成必要的头文件；在后期的开发过程中，则可用这些头文件来生成 **ninja** 或 **makefile** 等编译脚本。

kconfig：用於區分不同的平台並對代碼進行裁剪。在應用開發中，這主要體現在對各種驅動功能的開啟或關閉上。如下圖所示，驅動程序的實現中定義了與 **Wi-Fi** 相關的 **kconfig** 宏。使用者可以在 **prj.config** 檔案中啟用該宏，如此一來，相關的驅動功能就會被激活。



The image shows two screenshots of Zephyr project configuration files. The left screenshot shows the `prj.conf` file with the following configuration:

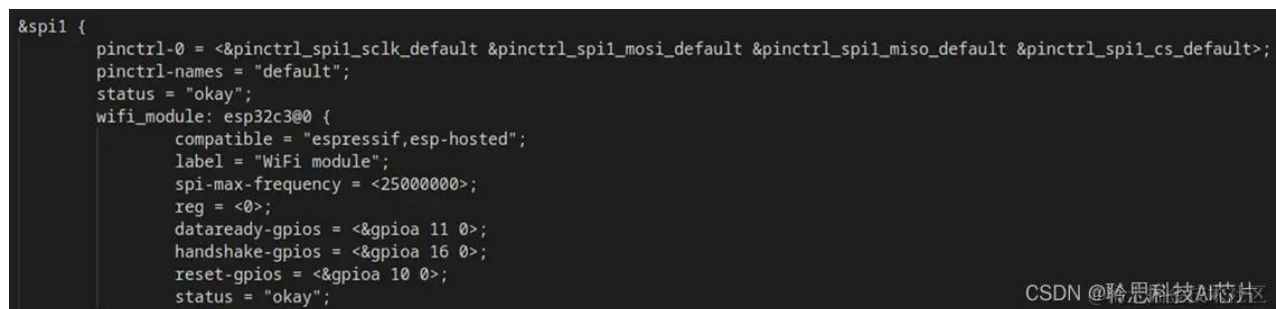
```
1 CONFIG_WIFI=y
2 CONFIG_CSK_WIFI_STATION=y
3 CONFIG_NET_L2_ETHERNET=y
4
5 CONFIG_NEWLIB_LIBC=y
```

The right screenshot shows the `CSK_WIFI_STATION` configuration in the `CSK_WIFI` directory:

```
30
31 config CSK_WIFI_STATION
32     bool "WiFi Station Enable"
33
34 config CSK_WIFI_SOFTAP
35     bool "WiFi SoftAP Enable"
36
```

Both screenshots are from the Zephyr project, specifically the `wifi_scan_ap` directory.

devicetree：用於配置硬體參數資訊。例如，對於 **WIFI** 模組，可以配置 **SPI** 腳位及傳輸速率，同時也能配置與之相關的同步腳位。



The image shows a snippet of a device tree source file for a WiFi module:

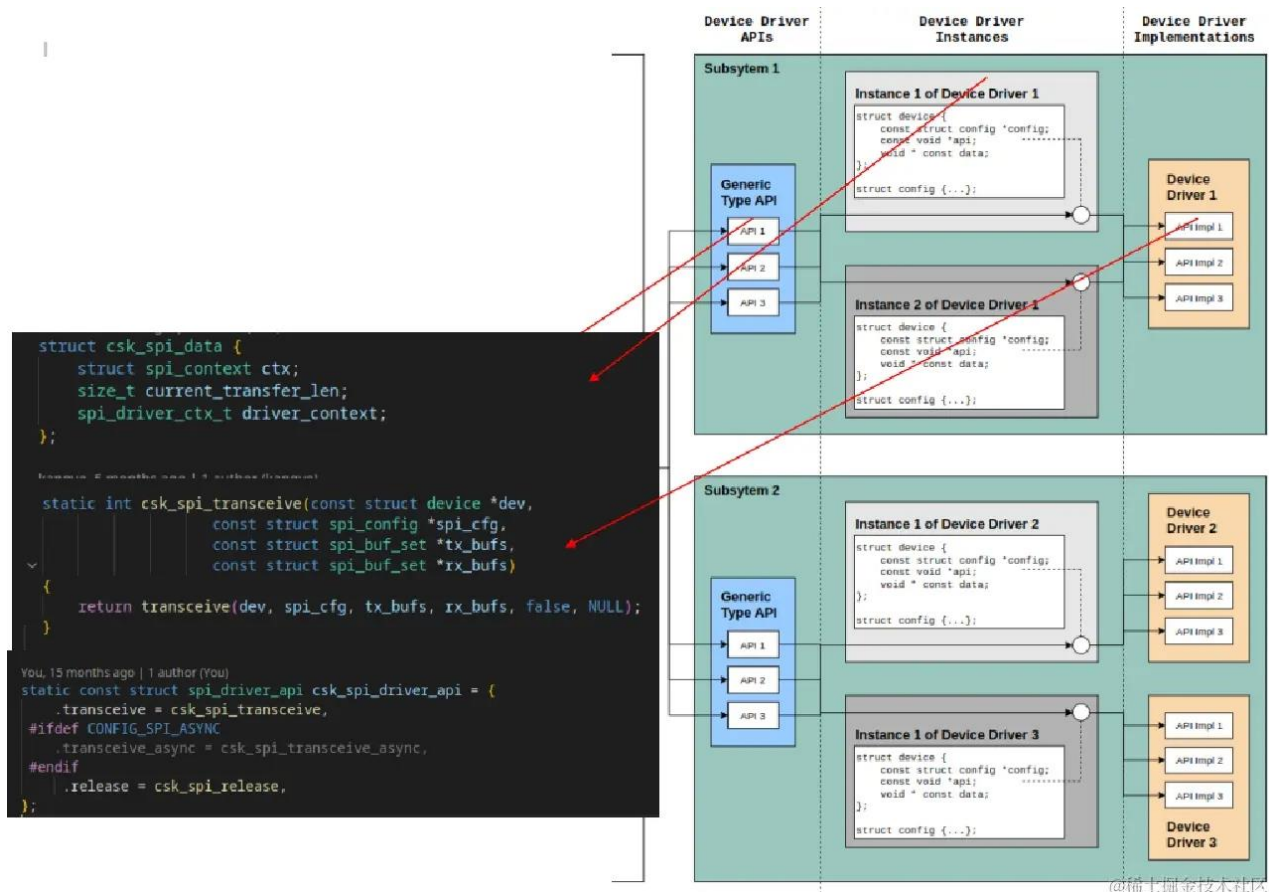
```
&spi1 {
    pinctrl-0 = <&pinctrl_spi1_sclk_default &pinctrl_spi1_mosi_default &pinctrl_spi1_miso_default &pinctrl_spi1_cs_default>;
    pinctrl-names = "default";
    status = "okay";
    wifi_module: esp32c3@0 {
        compatible = "espressif,esp-hosted";
        label = "WiFi module";
        spi-max-frequency = <25000000>;
        reg = <0>;
        dataready-gpios = <&gpioa 11 0>;
        handshake-gpios = <&gpioa 16 0>;
        reset-gpios = <&gpioa 10 0>;
        status = "okay";
    }
}
```

The snippet is from the `esp32c3` module, specifically the `wifi_module` node.

3.2 設備驅動模型

Zephyr 為幾乎所有的外設驅動程式都提供了統一的 **API** 接口。芯片原廠可根據這些 **API** 接口來開發自己的驅動程式。因此，使用者在使用各種外設時，可直接利用 **Zephyr** 所提供的接口來進行應用程式的開發。

Zephyr 的設備驅動都是在同一個地方進行統一初始化的。以 **SPI** 驅動為例，其宏函數其實就等價於這段代碼。這段代碼的作用就是註冊一個函數。該函數會在 **main** 函數之前被執行，其執行層級屬於應用程式層級。不過，驅動的執行層級會高於應用程式；層級越高，則執行順序就越靠前。無論如何，設備驅動都必須在 **main** 函數之前完成初始化。



3.3 線程調度與通訊

Zephyr 在線程調度方面的功能更為強大、靈活，能夠更好地滿足不同場景下的需求。

而 FreeRTOS 則更加簡單、易於使用，適合對資源需求較為簡單的嵌入式應用場景。

	Zephyr	FreeRTOS
线程创建	支持在编译期创建	只支持在运行时创建
调度	协作式与抢占式共存，可选且优先级可配的时间片轮询	协作式或抢占式内核，同等优先级时进行时间片轮询

Zephyr 與 FreeRTOS 線程通信對比：



- Semaphore
- Mutex
- Message Queue
- Events
- Pipe
- Fifo



- Binary semaphore
- Mutually exclusive semaphore
- Queue
- Event group
- Stream buffer
- ...

@福士掘金技术社区

Zephyr 提供了管道、消息隊列和信號量等多種線程通信機制，而 FreeRTOS 則提供了二值信號量、互斥量和隊列等線程通信機制。可見，Zephyr 所提供的線程通信機制更加多樣化。

3.4 記憶體管理

Zephyr 在內存管理方面擁有一些比其他 RTOS 更為先進的機制，如下所示：

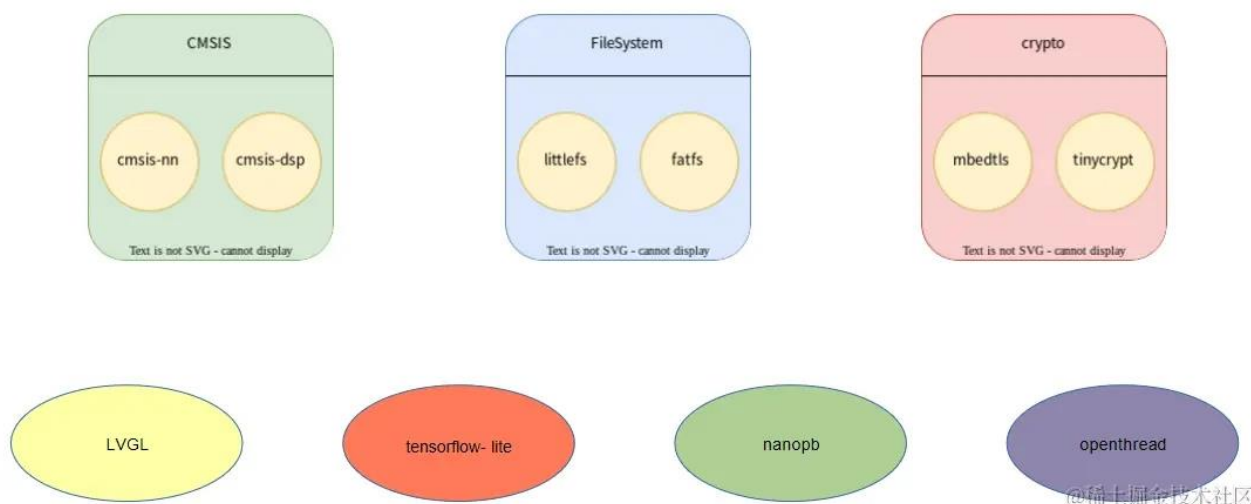
記憶體堆：提供基於堆的動態記憶體分配與釋放機制

Memory Slabs：提供預先分配一定數量內存塊的機制，並能夠快速分配和釋放內存塊。這種方式避免了堆內存管理的開銷，也減少了內存碎片的產生。

Memory Blocks Allocator：提供固定大小內存塊的動態分配與釋放機制，適用於需要頻繁分配、釋放同一大小內存塊的場景。

3.5 組件生態

越來越多的第三方組件庫已經加入或被移植進了 ZephyrProject 中，第三方組件的管理與接入流程也日益成熟。這為開發者節省了大量常用組件的移植與適配工作。



4. Zephyr 啟動流程

在此，我們以 `cortex_m` 為例，僅說明啟動流程。至於其他程式碼的運作流程，可參考：
blog.csdn.net/qq_36115224...

4.1 程序進入點

在 `include/zephyr/arch/arm/cortex_m/scripts/linker.ld` 中定义了 `ENTRY` 程序的入口點。

```
ENTRY(CONFIG_KERNEL_ENTRY)
```

這個宏的定義在 `Kconfig.zephyr` 中

```
config KERNEL_ENTRY
    string "Kernel entry symbol"
    default "__start"
    help
        Code entry symbol, to be set at linking phase.
```

`__start` 是程序的入口點，但這是 `gcc` 的規則。`Cortex-M` 系列處理器則是透過中斷向量表來確定程序的入口點。`Cortex-M` 系列處理器確定程序入口點的方式，就是透過中斷向量表來實現的。

在 `Cortex-M` 系列 MCU 中，如果設定為從 `flash` 啟動，`flash` 的前 1K 區域會用來儲存中斷向量表。

- 其中，第一个字是程序栈顶指针，用於初始化栈顶。
- 第二個字是復位向量，也就是 **ResetHandler** 的地址。這個地址被用作程序跳轉的目標地址，從而讓程序能夠跳轉到該地址處繼續運行。

詳細說明可參考異常模型。該機制為 **Cortex-M** 系列 **MCU** 中所獨有。相對地，不同的芯片在系統啟動時，對可執行程序都有一定的格式要求，必須在程序的頭部添加符合操作規範的標頭。

在中斷向量表中，**reset** 處理函式的地址定義在 `arch/arm/core/cortex_m/reset.S` 中。

```
SECTION_SUBSEC_FUNC(TEXT,_reset_section,z_arm_reset)
```

4.2 汇编初始化

在 `arch/arm/core/cortex_m/reset.S` 中

```
SECTION_SUBSEC_FUNC(TEXT,_reset_section,__start)

#if defined(CONFIG_DEBUG_THREAD_INFO)
    /* Clear z_sys_post_kernel flag for RTOS aware debuggers */
    movs.n r0, #0
    ldr r1, =z_sys_post_kernel
    strb r0, [r1]
#endif /* CONFIG_DEBUG_THREAD_INFO */

#if defined(CONFIG_INIT_ARCH_HW_AT_BOOT)
    /* 复位 CONTROL 寄存器，在特权模式和非特权模式均使用 MSP，
     * 当切换栈指针之后必须使用 ISB 指令刷新流水线，
     * 以保证在 ISB 之后执行的指令都使用新的栈
     */
    movs.n r0, #0
    msr CONTROL, r0
    isb
#endif

#if defined(CONFIG_CPU_CORTEX_M_HAS_SPLIM)
    /* 堆栈限制寄存器分别限制 MSP 和 PSP 寄存器可以下降的程度，此处设置为 0 */
    movs.n r0, #0
    msr MSPLIM, r0
    msr PSPLIM, r0
#endif /* CONFIG_CPU_CORTEX_M_HAS_SPLIM */
```

```

#endif /* CONFIG_INIT_ARCH_HW_AT_BOOT */

#if defined(CONFIG_PM_S2RAM)
    /* 低功耗相关初始化 */
    bl arch_pm_s2ram_resume
#endif /* CONFIG_PM_S2RAM */

#if defined(CONFIG_PLATFORM_SPECIFIC_INIT)
    /* 针对内存，cache，jtag，时钟，中断的一些特殊配置 */
    bl z_arm_platform_init
#endif

#if defined(CONFIG_INIT_ARCH_HW_AT_BOOT)
#if defined(CONFIG_CPU_HAS_ARM_MPU)
    /* 操作系统未运行之前使用平坦内存模型，所有内存均不受保护，
     * 为避免在初始化过程中触发读写保护进入异常，一定要关闭 MPU
     */
    movs.n r0, #0
    ldr r1, =_SCS_MPU_CTRL
    str r0, [r1]
    dsb

#endif /* CONFIG_CPU_HAS_ARM_MPU */
    /* ARM32 使用满递减栈，栈顶指针初始时刻指向高地址，
     * 将 MSP 指向 z_main_stack 的末尾，以便后续进行函数调用
     */
    ldr r0, =z_main_stack + CONFIG_MAIN_STACK_SIZE
    msr msp, r0

    /* 清除 MPU 配置，关闭所有中断，清除被挂起的中断，重置 Cache 配置等 */
    bl z_arm_init_arch_hw_at_boot
#endif /* CONFIG_INIT_ARCH_HW_AT_BOOT */

    /* 屏蔽中断 */
#if defined(CONFIG_ARMV6_M_ARMV8_M_BASELINE)
    cpsid i
#elif defined(CONFIG_ARMV7_M_ARMV8_M_MAINLINE)
    movs.n r0, #_EXC_IRQ_DEFAULT_PRI0
    msr BASEPRI, r0
#else
#error Unknown ARM architecture

```

```

#endif

#ifdef CONFIG_WDOG_INIT
    /* 开启看门狗 */
    bl z_arm_watchdog_init
#endif

#ifdef CONFIG_INIT_STACKS
    /* 将栈全部设置为 0xaa，可用于监测剩余栈容量 */
    ldr r0, =z_interrupt_stacks
    ldr r1, =0xaa
    ldr r2, =CONFIG_ISR_STACK_SIZE + MPU_GUARD_ALIGN_AND_SIZE
    bl z_early_memset
#endif

    /* 初始化 PSP，将 CONTROL 的 SPSEL 位设置为 1（在特权模式下使用 MSP，非特权模式下使用 PSP）
    * 后续操作将使用 z_interrupt_stacks 作为栈进行初始化
    */
    ldr r0, =z_interrupt_stacks
    ldr r1, =CONFIG_ISR_STACK_SIZE + MPU_GUARD_ALIGN_AND_SIZE
    adds r0, r0, r1
    msr PSP, r0
    mrs r0, CONTROL
    movs r1, #2
    orrs r0, r1 /* CONTROL_SPSEL_Msk */
    msr CONTROL, r0
    isb

    bl z_prep_c

```

4.3 C 語言的初始化

z_prep_c()函數的定義在 arch/arm/core/cortex_m/prep_c.c 中

```

void z_arm_prep_c(void)
{
    relocate_vector_table();
#ifdef CONFIG_CPU_HAS_FPU
    z_arm_floating_point_init();

```

```

#endif

    z_bss_zero();
    z_data_copy();
#if ((defined(CONFIG_ARMV7_R) || defined(CONFIG_ARMV7_A)) &&
defined(CONFIG_INIT_STACKS))
    z_arm_init_stacks();
#endif

    z_arm_interrupt_init();
    z_cstart();
    CODE_UNREACHABLE;
}

```

對於不同的平台，還需要進行一些額外的操作。例如，必須對浮點寄存器進行初始化，並重新設定中斷向量表的位置。在系統啟動之前，還會執行一段芯片內置的程序。該程序會將中斷向量表的位置設定為 0x00000000。當程序從啟動模式跳轉到指定的存儲器區域後，首先必須立即關閉中斷，以避免中斷的發生以及程序跳轉到錯誤的中斷處理函數中。在重新設定中斷向量表的偏移位置之後，再重新啟動中斷即可。

初始化內核並啟動任務調度

```

FUNC_NO_STACK_PROTECTOR
FUNC_NORETURN void z_cstart(void)
{
    /* 代碼覆蓋率測試相關 */
    gcov_static_init();

    /* 調用初始化級別為 INIT_LEVEL_EARLY 的函數進行初始化 */
    z_sys_init_run_level(INIT_LEVEL_EARLY);

    /* z_arm_interrupt_stack_setup 初始化 MSP
    * z_arm_exc_setup 初始化 PENDSV、SysTick 等中斷的優先級，其中 PENDSV 優先級為最低
    * z_arm_fault_init 初始化 Fault 中斷。
    * z_arm_cpu_idle_init 初始化 idle 線程
    * z_arm_clear_faults 清除所有故障標志
    * z_arm_mpu_init 初始化 MPU
    * z_arm_mmu_init 初始化 MMU
    */
    arch_kernel_init();

    /* 日誌初始化 */
    LOG_CORE_INIT();
}

```

```

#if defined(CONFIG_MULTITHREADING)
    /* Note: The z_ready_thread() call in prepare_multithreading() requires
     * a dummy thread even if CONFIG_ARCH_HAS_CUSTOM_SWAP_TO_MAIN=y
     */

    struct k_thread dummy_thread;

    z_dummy_thread_init(&dummy_thread);
#endif

    /* 初始化驱动中的静态节点 */
    z_device_state_init();

    /* 其他的硬件初始化 */
    z_sys_init_run_level(INIT_LEVEL_PRE_KERNEL_1);
    z_sys_init_run_level(INIT_LEVEL_PRE_KERNEL_2);

#ifdef CONFIG_STACK_CANARIES
    /* CONFIG_STACK_CANARIES 用于开启堆栈金丝雀功能，这是一种安全特性，有助于监测堆栈溢出，
     * 当启动该功能时，系统启动时会生成一个随机数并保存在 __stack_chk_guard 中，
     * 在函数返回之前会检查该值确保它没有被缓冲区溢出所覆盖。
     */

    uintptr_t stack_guard;

    z_early_boot_rand_get((uint8_t *)&stack_guard, sizeof(stack_guard));
    __stack_chk_guard = stack_guard;
    __stack_chk_guard <= 8;
#endif /* CONFIG_STACK_CANARIES */

#ifdef CONFIG_TIMING_FUNCTIONS_NEED_AT_BOOT
    /* timing_init 函数用于初始化系统计时器 */
    timing_init();
    timing_start();
#endif

#ifdef CONFIG_MULTITHREADING
    /* CONFIG_MULTITHREADING 为 y 时，使用多线程，否则只会会有一个 main 线程，
     * 默认情况下都启用多线程，通过将 main 线程添加到就绪队列中然后开启任务调度
     */

    switch_to_main_thread(prepare_multithreading());
#else
#ifdef ARCH_SWITCH_TO_MAIN_NO_MULTITHREADING

```

```

    /* Custom ARCH-specific routine to switch to main()
     * in the case of no multi-threading.
     */
    ARCH_SWITCH_TO_MAIN_NO_MULTITHREADING(bg_thread_main,
        NULL, NULL, NULL);
#else
    bg_thread_main(NULL, NULL, NULL);

    /* LCOV_EXCL_START
     * We've already dumped coverage data at this point.
     */
    irq_lock();
    while (true) {
    }
    /* LCOV_EXCL_STOP */
#endif
#endif /* CONFIG_MULTITHREADING */

    /*
     * Compiler can't tell that the above routines won't return and issues
     * a warning unless we explicitly tell it that control never gets this
     * far.
     */

    CODE_UNREACHABLE; /* LCOV_EXCL_LINE */
}

```

進入 `z_cstart` 之後，首先會調用與平台相關的內核初始化函數，接著再利用模組自動初始化機制，根據不同的優先級來調用相關的初始化函數。需要注意的是，相同優先級的模組之間不應該存在依賴關係，被依賴的模組應該先被初始化。在調度器啟動之前，會先調用優先級為 `INIT_LEVEL_PRE_KERNEL_1` 和 `INIT_LEVEL_PRE_KERNEL_2` 的初始化函數。此時調度器尚未運行，因此這些函數中不應該使用操作系統所提供的功能。初始化完成之後，會將主執行緒加入就緒隊列並啟動調度器，此時主執行緒會從 `bg_thread_main` 函數開始運行。

4.4 主线程

`bg_thread_main` 的定義在 `kernel/init.c` 中。

```

static void bg_thread_main(void *unused1, void *unused2, void *unused3)
{
    ARG_UNUSED(unused1);
    ARG_UNUSED(unused2);
    ARG_UNUSED(unused3);

#ifdef CONFIG_MMU
    /* Invoked here such that backing store or eviction algorithms may
     * initialize kernel objects, and that all POST_KERNEL and later tasks
     * may perform memory management tasks (except for z_phys_map() which
     * is allowed at any time)
     */
    z_mem_manage_init();
#endif /* CONFIG_MMU */
    z_sys_post_kernel = true;

    /* 调用优先级为 INIT_LEVEL_POST_KERNEL 的初始化函数，
     * 此时内核已经开始运行，可以使用操作系统 API
     */
    z_sys_init_run_level(INIT_LEVEL_POST_KERNEL);
#if CONFIG_STACK_POINTER_RANDOM
    z_stack_adjust_initialized = 1;
#endif

    /* 从控制台输出系统启动标识 */
    boot_banner();

#if defined(CONFIG_CPP)
    /* 初始化 CPP 运行环境 */
    void z_cpp_init_static(void);
    z_cpp_init_static();
#endif

    /* 调用优先级为 INIT_LEVEL_APPLICATION 的初始化函数 */
    z_sys_init_run_level(INIT_LEVEL_APPLICATION);
    /* Zephyr 支持静态创建线程，线程对应的信息在编译时确定，
     * 随代码一起被编译到程序中，系统启动之后从对应地址将线程的信息从 flash 中读出，
     * 创建并初始化线程并将其添加到就绪队列中等待操作系统调度。
     */
    z_init_static_threads();

#ifdef CONFIG_KERNEL_COHERENCE

```

```

    __ASSERT_NO_MSG(arch_mem_coherent(&_kernel));
#endif

#ifdef CONFIG_SMP
    if (!IS_ENABLED(CONFIG_SMP_BOOT_DELAY)) {
        z_smp_init();
    }
    z_sys_init_run_level(INIT_LEVEL_SMP);
#endif

#ifdef CONFIG_MMU
    z_mem_manage_boot_finish();
#endif /* CONFIG_MMU */

#ifdef CONFIG_CPP_MAIN
    extern int main(void);
#else
    extern void main(void);
#endif

    /* 跳转到 main 函数 */
    (void)main();

    /* Mark nonessential since main() has no more work to do */
    z_main_thread.base.user_options &= ~K_ESSENTIAL;

#ifdef CONFIG_COVERAGE_DUMP
    /* Dump coverage data once the main() has exited. */
    gcov_coverage_dump();
#endif
}

```

其中包含幾個系統運行時的重要操作：配置 MMU（如果存在）；調用優先級為 `INIT_LEVEL_POST_KERNEL` 的初始化函數；初始化 CPP 運行環境；調用優先級為 `INIT_LEVEL_APPLICATION` 的初始化函數；創建通過宏靜態生成的線程，並將其添加到就緒隊列中；初始化對稱多核處理器（如果存在多個處理器且多核調度功能已啟用）。在這些準備工作完成後，系統會跳轉到用戶編寫的 `main` 函數中。如果 `main` 函數執行完畢並返回，最終系統會回到 `z_thread_entry` 點，然後被銷毀。

學習方法

1. 查看官方網站：[docs.zephyrproject.org/latest/intr...](https://docs.zephyrproject.org/latest/intro/)
2. 在 bilibili 上觀看影片：search.bilibili.com/all?keyword...
3. CSDN、知乎之類的網站上也有不少資料，可以自己去搜尋。

參考資料：

1. [Zephyr 入門，看這篇就夠了（含 FreeRTOS 對比）](#)
2. blog.csdn.net/qq_36115224...
3. lgl88911.github.io/2020/11/29/...

後記：

Zephyr RTOS 在網上有很多相關資料，大家可以直接去搜尋學習。這也說明了 Zephyr RTOS 相當受歡迎。各種硬體也都支援 Zephyr RTOS。它的功能相當豐富，如果自己的專案缺少某種功能，就可以從網上將該功能移植過來。例如，gcov 代碼覆蓋率測試模組，一般的 RTOS 都不支援這個功能，但可以將其移植到 Zephyr RTOS 上。

在國內，操作系統可能還相當陌生，但在歐美，尤其是在各所大學中，各種定製版或學術版的操作系統不斷出現。現在，似乎人人都能打造操作系統了。這些小眾操作系統未必能流行起來，但每個都有其獨特之處。在某些特定場景下，小眾操作系統甚至可能帶來更高的效率與更好的效果。開源操作系統的領域，本來就是各種創意不斷被模仿的領域啊。